

De la implementación al entendimiento

Resumen técnico de lo implementado y de todo lo revisado sobre monitorización en Azure —
preparado como material de repaso para la entrevista técnica del 24 de agosto con Daniel Bautista (ERNI / B.Braun).

1. Qué se implementó en el código

Se añadió instrumentación de Application Insights a las dos aplicaciones .NET del proyecto — `Alakai.FestivalManager.Api` y `Alakai.FestivalManager.Admin` — apuntando ambas al mismo recurso de Azure, `appi-alakai-swimout`. La implementación pasó por dos versiones: la primera se instaló, se detectó que era la incorrecta para el paquete que realmente se había descargado, y se corrigió antes de dar nada por bueno. Vale la pena documentar las dos, porque el motivo del cambio es en sí mismo un buen ejemplo para la entrevista.

LO QUE PASÓ REALMENTE

Al instalar `Microsoft.ApplicationInsights.AspNetCore` sin fijar versión, NuGet trajo la 3.1.2 — y en la serie 3.x de ese paquete, Microsoft **eliminó** `ITelemetryInitializer` **de la API pública**. No fue un error de código: reescribieron el SDK entero sobre **OpenTelemetry** (el estándar de observabilidad, el mismo que sale en la pregunta 6 de la guía de entrevista) con el exportador de Azure Monitor por debajo, en vez del pipeline propietario que tenía la v2.x. El fallo de compilación al añadir el `ITelemetryInitializer` fue la señal — se confirmó la causa contra la documentación oficial de Microsoft antes de tocar nada más, y se corrigió sustituyendo el paquete entero por el correcto en vez de forzar una versión antigua.

Paquete NuGet instalado (versión final, correcta)

PAQUETE	PROYECTO(S)	PARA QUÉ
<code>Azure.Monitor.OpenTelemetry.AspNetCore</code>	Api y Admin	SDK de telemetría basado en OpenTelemetry — auto-instrumenta requests, dependencias y excepciones, y las exporta al endpoint de Azure Monitor
<code>Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore</code>	Api	Expone <code>/health</code> comprobando la conexión a la base de datos (este paquete no cambió)

Etiquetado por servicio: sin ficheros nuevos

Como las dos apps comparten el mismo recurso de Application Insights, hace falta distinguir qué telemetría viene de la Api y cuál del Admin para que el **Application Map** las dibuje como dos nodos separados. En el planteamiento inicial (v2.x) esto se hacía con una clase nueva por proyecto (`CloudRoleTelemetryInitializer.cs`); esos ficheros se llegaron a crear pero se **borraron** al migrar a OpenTelemetry, porque ese patrón ya no existe. En la versión final, el nombre del servicio se declara directamente en `Program.cs` como un atributo de "Resource" de OpenTelemetry (`service.name`) — sin ningún fichero adicional.

Registro en `Program.cs` (versión final)

```
// Api
builder.Services.AddOpenTelemetry()
    .UseAzureMonitor(options =>
    {
        options.ConnectionString = builder.Configuration["ApplicationInsights:ConnectionString"];
    })
    .ConfigureResource(resourceBuilder => resourceBuilder.AddAttributes(new Dictionary<string, object>
    {
        { "service.name", "FestivalManager-Api" }
    }));

// Admin — mismo patrón, con "service.name" = "FestivalManager-Admin"

// Solo en la Api, además (esto no cambió):
builder.Services.AddHealthChecks().AddDbContextCheck<FestivalManagerDbContext>();
...
app.MapHealthChecks("/health");
```

El endpoint `/health` existe para que Azure Monitor **Availability Tests** pueda hacer ping periódico desde fuera y confirmar que la Api responde y tiene conexión a base de datos, antes de que lo note un usuario real. `appsettings.json` no se tocó al migrar de paquete — sigue leyéndose la misma clave de configuración, solo cambió el SDK que la consume.

Limpieza de un agente duplicado en Azure

Al revisar el App Service en el portal aparecían dos variables de entorno heredadas de un intento anterior: `ApplicationInsightsAgent_EXTENSION_VERSION` y `APPLICATIONINSIGHTS_CONNECTION_STRING`. La primera activa el **agente de auto-instrumentación sin código** de Azure App Service (el botón "Turn on Application Insights" del propio portal), y apuntaba a un recurso ya borrado — quedaba huérfana. Mantenerla habría significado dos pipelines de telemetría compitiendo a la vez: el agente automático del App Service y el SDK de OpenTelemetry ya integrado en el código. Se eliminaron ambas variables y se sustituyeron por una sola, con nombre deliberadamente distinto para no confundirla con el agente antiguo: `ApplicationInsights__ConnectionString` (doble guion bajo), apuntando a `appi-alakai-swimout`. Resultado: una única fuente de verdad — el código, no un agente en la sombra.

La connection string nunca se commitea

DECISIÓN DE DISEÑO DELIBERADA

`appsettings.json` queda con `"ApplicationInsights": { "ConnectionString": "" }` — vacío a propósito. El valor real se configura en dos sitios distintos según el entorno, nunca en el repositorio:

- **Local:** `dotnet user-secrets set "ApplicationInsights:ConnectionString" "..."`, en cada proyecto por separado.
- **Azure:** como Application Setting del App Service, con doble guion bajo:

```
ApplicationInsights__ConnectionString.
```

La connection string nunca se commitea

DECISIÓN DE DISEÑO DELIBERADA

appsettings.json queda con "ApplicationInsights": { "ConnectionString": "" } — vacío a propósito. El valor real se configura en dos sitios distintos según el entorno, nunca en el repositorio:

- **Local:** `dotnet user-secrets set "ApplicationInsights:ConnectionString" "..."`, en cada proyecto por separado.
- **Azure:** como Application Setting del App Service, con doble guion bajo:

```
ApplicationInsights__ConnectionString.
```

2. El concepto que generó más confusión: "local" no significa "sin datos en Azure"

"Local" describe únicamente **dónde se ejecuta el proceso** (tu portátil), no dónde se guarda la telemetría. El SDK de Application Insights no almacena nada localmente: en cuanto procesa una petición, la manda por red al endpoint de ingesta de Azure, usando la connection string configurada. Esa connection string apunta al mismo recurso `appi-alakai-swimout` tanto si el proceso corre en Windows local como en el App Service — el destino es siempre el mismo.

Consecuencia práctica: la telemetría generada en local y la de producción, más adelante, se van a mezclar en el mismo recurso. Cada evento queda etiquetado con `cloud_RoleInstance` (el nombre de la máquina) — en local, tu hostname de Windows; en Azure, el nombre de la instancia del App Service — lo que permite diferenciarlas si hace falta.

3. El portal de Azure, panel por panel

PANEL	PARA QUÉ SIRVE	CUÁNDO USARLO
Live Metrics	Stream en tiempo real (sub-segundo) de peticiones/seg, fallos, latencia y excepciones	Mientras pruebas algo AHORA MISMO, no para histórico
Failures	Excepciones y peticiones fallidas agrupadas por operación, con stack trace de ejemplos concretos	Investigar por qué algo está roto
Performance	Latencia por endpoint y por dependencia (SQL, HTTP saliente); permite drill-down a "samples" y al detalle de transacción end-to-end	Saber qué endpoint es lento y por qué
Application Map	Diagrama automático de qué componente llama a cuál, con latencia y % de fallo por conexión; resalta en rojo las anomalías	Vista rápida de salud general, una vez por semana
Logs (KQL)	Todos los datos en bruto, consultables con Kusto Query Language, incluidas métricas técnicas de bajo nivel (ej. <code>http.client.active_requests</code>)	Investigación profunda o correlación entre señales — no para el día a día
Metrics	Constructor de gráficos personalizados sobre métricas	Cuando ya sabes qué quieres

Detalle de Performance: cómo leerlo bien

- El toggle **Server** (frente a *Browser*) mide desde el backend, no desde el navegador del usuario.
- `CONNECT /_blazor` con duraciones de varios minutos **no es una petición lenta** — es la conexión WebSocket persistente de Blazor Server; ese número es cuánto tiempo estuvo la conexión abierta, no latencia. Se ignora al buscar cuellos de botella reales.
- Cambiar de **Avg** a **95th percentile** es importante: la media esconde picos raros, el percentil 95 muestra cómo lo vive el usuario con peor suerte.
- Un clic en una operación lenta abre samples concretos; desde un sample, el **end-to-end transaction detail** da el timeline exacto de en qué se fue el tiempo (código propio, SQL, dependencia externa).

4. Detección automática (Smart Detection) vs. alertas manuales

Existen dos capas distintas para enterarte de que algo va mal, y es la distinción más importante de todo lo revisado:

CAPA 1 — SMART DETECTION (YA ACTIVA POR DEFECTO)

Diez detectores basados en machine learning, activos desde que se creó el recurso, sin configurar nada. Comparan el comportamiento actual contra el histórico *de tu propia app* — no contra un número fijo que tú definas. Cubren: tiempo de carga/respuesta lento, degradación de respuesta y de dependencias, subida de excepciones, posible fuga de memoria, patrones de seguridad sospechosos y subida anómala de volumen de datos. Notifican por email automáticamente a los Owners/Contributors de la suscripción.

CAPA 2 — ALERTAS MANUALES (AZURE MONITOR ALERTS)

Para cuando SÍ importa un número de negocio concreto (ej. un SLA de "checkout nunca > 3s"). Se crean explícitamente sobre un **signal** (métrica, log query, evento) con una condición, y se notifican a través de un **Action Group** — un recurso de notificación reutilizable (email/SMS/etc.) que se define una vez y se reutiliza en todas las alertas futuras, en vez de repetir el email en cada regla.

En la práctica, el flujo de trabajo no es "vigilar métricas activamente" — es "si algo está mal, me llega un email (de Smart Detection o de una alerta manual), y entonces abro el portal a investigar ese aviso concreto" usando Failures/Performance/Application Map.

5. La alerta creada: Failed requests + Dynamic threshold

Se intentó primero crear una alerta sobre el signal **Smart Detector → Failure Anomalies**, pero ese signal no apareció disponible en el asistente de "Create an alert rule" para este recurso. Se sustituyó por un signal equivalente que sí estaba disponible: **Metrics → Failed requests**, con las siguientes decisiones de configuración:

CAMPO	VALOR ELEGIDO	POR QUÉ
Threshold type	Dynamic (no Static)	Azure aprende el rango normal de fallos de la app y avisa solo ante desviaciones — igual que Smart Detection, sin que haya que adivinar un número fijo
Operator	Greater than	Solo interesa cuando SUBEN los fallos, no cuando bajan
Threshold sensitivity	Medium	Punto de partida razonable; Low = menos avisos, High = más sensible
Aggregation type	Count	Cuenta el número de peticiones fallidas en la ventana evaluada
Scope	Recurso <code>appi-alakai-swimout</code> completo	Suma los fallos de Api y Admin juntos (ver limitación abajo)
Action Group	Nuevo, con notificación por email	Reutilizable en cualquier alerta futura sin repetir el email
Severity	Sev 2 - Warning	Punto medio: ni se ignora, ni suena a incendio en cada aviso

Qué monitoriza exactamente "Failed requests"

- Solo **peticiones HTTP entrantes** procesadas por Api o Admin (lado servidor) — no incluye llamadas salientes (SQL, Google Analytics), esas viven en la métrica hermana *Dependency call failures*.
- Se marca como fallo cualquier respuesta con código **4xx o 5xx**, o cualquier petición donde salte una excepción sin capturar, independientemente del código devuelto.

LIMITACIÓN A TENER EN CUENTA

No distingue "fallo real" de "fallo esperado por negocio" — un `404` devuelto porque un festival no existe cuenta igual que un `500` real. Y como el Scope es el recurso completo, la alerta no dice por sí sola si el problema es de la Api o del Admin — para eso hay que abrir Failures y filtrar. Ambas cosas se pueden refinar más adelante con *Split by dimensions* (por `cloud_RoleName`, por operación) si hiciera falta más granularidad.

6. Conexión con las preguntas de la entrevista

Este trabajo cierra directamente tres de los huecos identificados en la preparación (preguntas 6-8 sobre monitorización):

PREGUNTA	RESPUESTA AHORA RESPALDADA POR EXPERIENCIA PROPIA
----------	---

Logs vs. métricas vs. trazes	Ya no es solo teoría: Application Insights da las tres cosas a la vez desde un único SDK — logs de <code>ILogger<T></code> ya existentes, métricas como <i>Failed requests</i> , y trazes distribuidos visibles en el Application Map, correlacionados por <code>operation_Id</code> .
Qué usarías en Azure para monitorizar una app .NET	Application Insights + Log Analytics (KQL) + Azure Monitor Alerts + App Service diagnostics + Availability Tests — y ahora los cinco están, como mínimo, activados y probados en este proyecto, no solo mencionados de memoria.
Qué es Application Insights	Se puede describir con ejemplos vividos en primera persona: Live Metrics viendo el propio tráfico local en directo, Application Map mostrando Admin → Api → SQL con el nombre de cada servicio, y una alerta real creada con Dynamic threshold y Action Group.

Frases de apoyo para la entrevista

TRANSICIÓN LOGS → APPLICATION INSIGHTS

"Ya tenía logging estructurado con `ILogger<T>` en capas correctas — middleware de excepciones, eventos de negocio con placeholders estructurados. El siguiente paso natural era Application Insights, porque añade métricas y trazes distribuidos sin cambiar cómo logueo, y lo he probado de extremo a extremo: instrumentación, portal, y una alerta real con umbral dinámico."

SOBRE DETECCIÓN AUTOMÁTICA VS. ALERTAS MANUALES

"Azure ya trae diez detectores de anomalías activos por defecto que aprenden el comportamiento normal de la app con machine learning, sin que yo defina umbrales. Para lo que sí depende de un número de negocio concreto, creé una alerta manual con umbral dinámico sobre *Failed requests*, con un Action Group reutilizable para no repetir la configuración de notificación en cada regla nueva."

ANÉCDOTA DE DEPURACIÓN REAL — EL CAMBIO DE PAQUETE

"Al instalar Application Insights me encontré con que Microsoft había reescrito el SDK entero sobre OpenTelemetry en la versión 3, eliminando la API que estaba usando. En vez de fijar una versión antigua para que compilara, investigué el motivo del breaking change contra la documentación oficial y migré al patrón nuevo — que además es más alineado con el estándar de la industria. Es un ejemplo real de cómo reacciono cuando una dependencia cambia debajo de mí: entender el porqué antes de parchear el síntoma."

7. Estado final: qué queda pendiente

- Confirmar en el Action Group que el email de destino es el correcto (hecho durante esta sesión de trabajo).
- Considerar separar el `cloud_RoleName` en local (ej. `swimout-local-dev`) para no mezclar telemetría de pruebas con la de producción una vez se despliegue.
- Opcionalmente, crear una segunda alerta sobre *Dependency call failures* para cubrir fallos en llamadas salientes (SQL, Google Analytics), que *Failed requests* no cubre directamente.
- No es necesario implementar nada más en código antes de la entrevista — el objetivo de esta sesión era entender y poder explicar lo ya construido, no ampliarlo.